

INSTITUTO INFNET
ESCOLA SUPERIOR DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE SOFTWARE



Projeto de Bloco: Sistemas Digitais Embarcados

[26E2_2]

TP3

ALUNO: João Paulo Fonseca de Araújo

E-MAIL: joao.paraujo@al.infnet.edu.br

1. Introducao

Este relatorio documenta o desenvolvimento do TP3, que integra tres frentes de trabalho:

(A) projeto e validacao de uma maquina de estados finitos (FSM) em Verilog, sintetizada e testada fisicamente na placa Tang Nano 4K; (B) implementacao de estruturas de controle avancadas em Assembly ARM64, incluindo loops, condicionais multiplas, tabela de salto e um mini-parser de comandos de texto; (C) mapeamento dos registradores GPIO do System-on-Chip BCM2837/BCM2710A1 do Raspberry Pi Zero 2W, com demonstracao de acesso via instrucoes LDR/STR; (D) automacao de build via Makefile modular; e (E) documentacao da arquitetura integrada do projeto.

Uma restricao de ambiente relevante orienta todas as decisoes tecnicas descritas a seguir: nao ha disponibilidade do hardware Raspberry Pi Zero 2W fisico para este projeto. Por esse motivo, toda a frente de Assembly ARM64 (Etapas B e C) foi desenvolvida e validada exclusivamente por emulacao com QEMU (qemu-aarch64), com depuracao via GDB remoto e analise estatica via objdump. Sempre que um requisito do enunciado dependeria estritamente do hardware fisico do Raspberry Pi, essa adaptacao esta sinalizada explicitamente no texto correspondente. Ja a frente de FPGA (Etapa A) foi integralmente sintetizada e validada na placa Tang Nano 4K fisica, com evidencias reais de funcionamento.

1. FSM em Verilog (Tang Nano 4K)

1.1 Descricao da FSM projetada

Foi projetada uma FSM Receptora de Comandos (modulo fsm_control), concebida como ponte conceitual para a futura comunicacao ARM-FPGA: ela hoje reage aos botoes fisicos onboard da placa, mas ja incorpora o mesmo tipo de tratamento de sincronizacao e handshake que sera necessario quando o sinal de comando vier, no futuro, do GPIO do Raspberry Pi. A FSM possui seis estados: S_IDLE (aguardando comando), S_SYNC_CAPTURE (janela de estabilizacao do sinal capturado), S_VALIDATE (verifica se o comando e valido), S_EXECUTE (aplica o comando), S_ERROR (comando invalido) e S_WAIT_RELEASE (aguarda a liberacao do botao antes de retornar ao estado inicial, evitando reprocessamento indevido).

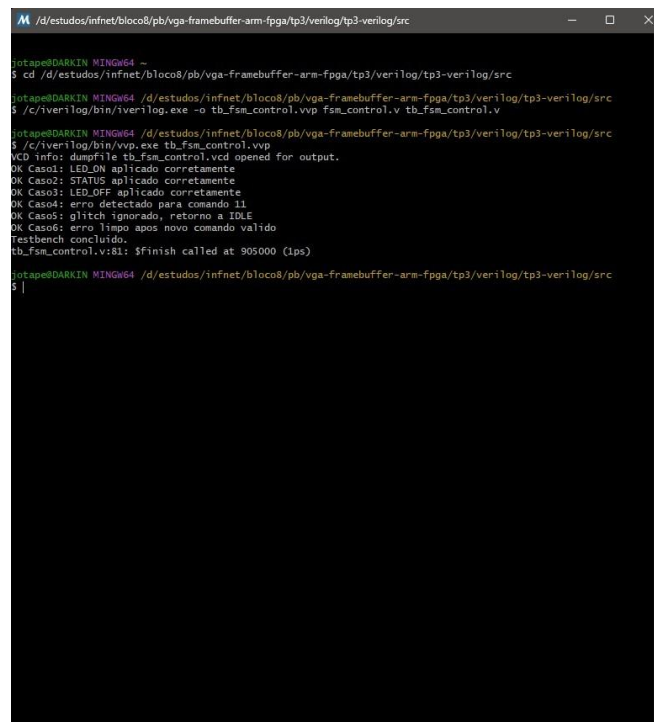
O protocolo de comando foi adaptado para usar os dois botoes fisicos disponiveis na Tang Nano 4K (USR_KEY_1, pino 14, e USR_KEY_2, pino 15), ja que a placa nao possui chaves externas nem botao de reset dedicado separado. USR_KEY_2 avanca um contador interno de dois bits que cicla entre os comandos LED_OFF, LED_ON, STATUS e um comando invalido (usado propositalmente para exercitar o estado S_ERROR); USR_KEY_1 confirma o comando atual e dispara a FSM. O reset do sistema e gerado internamente por um circuito de Power-On Reset (modulo por_reset), eliminando a necessidade de um pino de reset fisico.

1.2 Arquitetura hierarquica

O projeto foi organizado em seis modulos Verilog, mantendo a separacao entre logica de transicao de estado e logica de saida, conforme praticado no TP2: sync2ff.v (sincronizador de dois flip-flops para tratar metaestabilidade), debounce.v (filtro de ruido mecanico dos botoes, janela de aproximadamente 20 ms), por_reset.v (gerador de reset interno), fsm_control.v (a maquina de estados), blink_gen.v (gerador dos padroes visuais do LED) e top.v (modulo de topo que interliga os demais).

1.3 Testbench e simulacao

O testbench tb_fsm_control.v cobre seis casos de transicao distintos: aplicacao dos tres comandos validos (LED_ON, STATUS, LED_OFF), deteccao de comando invalido, um cenario de glitch (sinal de trigger mantido por tempo inferior ao necessario para captura estavel, validando a robustez da sincronizacao) e a recuperacao apos um erro. Todos os seis casos foram executados com sucesso via Icarus Verilog.



```
potape@DARKIN MINGW64 ~
$ cd /d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/verilog/tp3-verilog/src
potape@DARKIN MINGW64 /d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/verilog/tp3-verilog/src
$ ./c:/iverilog/bin/iverilog.exe -o tb_fsm_control.vvp fsm_control.v tb_fsm_control.v
potape@DARKIN MINGW64 /d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/verilog/tp3-verilog/src
$ ./c:/iverilog/bin/vvp.exe tb_fsm_control.vvp
VCD info: dumpFile tb_fsm_control.vcd opened for output.
OK Caso1: LED_ON aplicado corretamente
OK Caso2: STATUS aplicado corretamente
OK Caso3: LED_OFF aplicado corretamente
OK Caso4: erro detectado para comando 11
OK Caso5: glitch ignorado, retorno a IDLE
OK Caso6: erro limpo apos novo comando valido
Testbench concluido.
tb_fsm_control.v:81: $Finish called at 905000 (1ps)
potape@DARKIN MINGW64 /d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/verilog/tp3-verilog/src
$
```

Figura A1 - Compilacao (iverilog) e execucao (vvp) do testbench tb_fsm_control.v, confirmando sucesso nos seis casos de teste (OK Caso1 a OK Caso6).

As formas de onda geradas foram inspecionadas no GTKWave, confirmando visualmente as transicoes esperadas entre estados: o sinal cmd_data_sync percorre os valores 00, 01, 10 e 11 ao longo da simulacao, o led_result acompanha corretamente os comandos validos aplicados, e o error_latch sobe exatamente no trecho em que o comando invalido (11) e recebido, confirmando a entrada correta no estado S_ERROR.

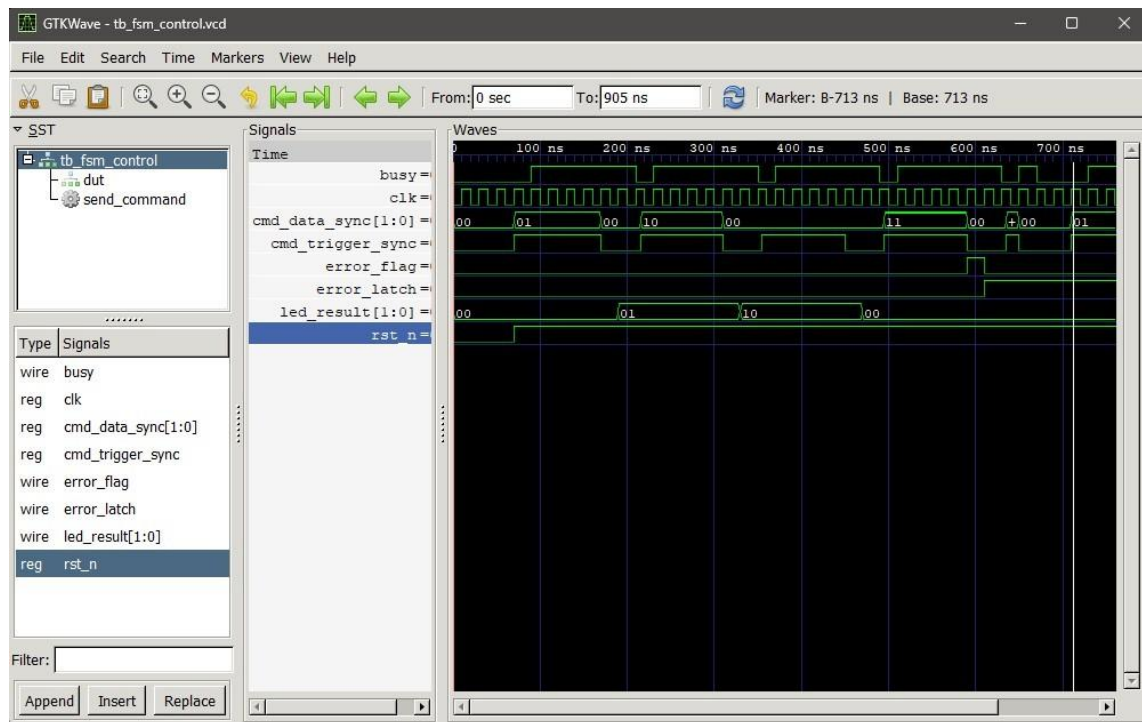


Figura A2 - Waveform da simulacao no GTKWave, mostrando cmd_data_sync, cmd_trigger_sync, error_flag, error_latch e led_result ao longo do tempo.

1.4 Sintese e validacao fisica

O projeto foi sintetizado no Gowin EDA para o dispositivo GW1NSR-LV4CQN48PC6/I5 (Tang Nano 4K), apos a correcao do pino do sinal de clock de 27 MHz para o pino 45 (identificado como o pino correto do oscilador onboard, em substituicao a um valor incorreto inicialmente utilizado). O log de sintese confirma zero erros, restando apenas avisos esperados e nao bloqueantes: truncamento de constante no modulo debounce.v e avisos relacionados ao roteamento do sinal de clock, que nao afetam a corretude funcional do projeto.

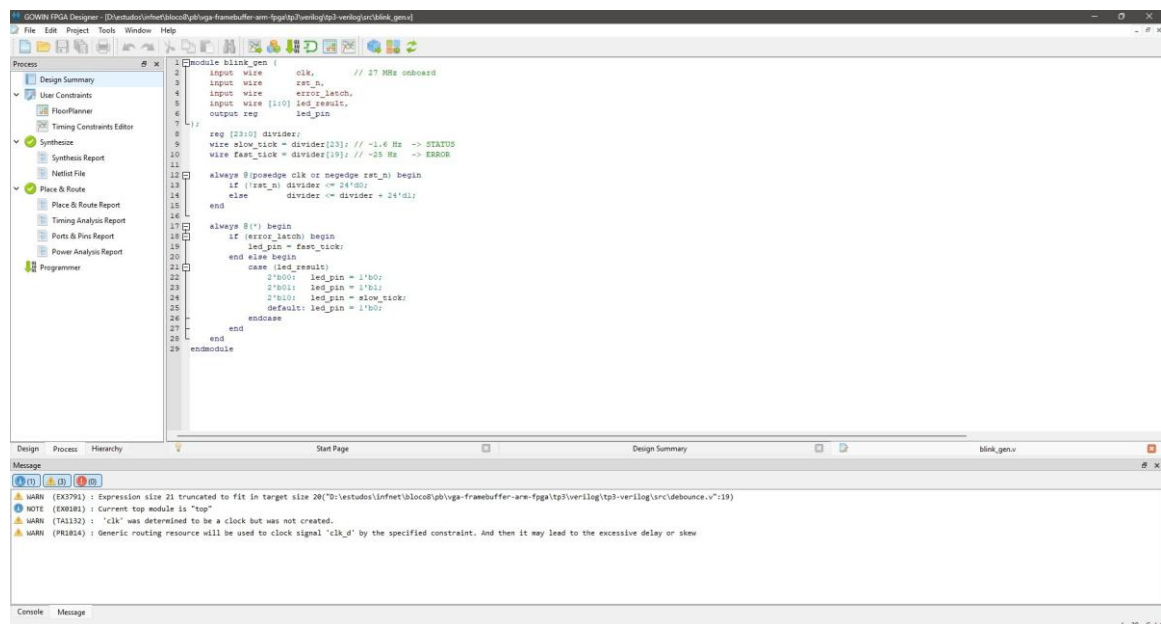


Figura A3 - Log de sintese do Gowin EDA sem erros (0 erros, 3 avisos nao bloqueantes), para o modulo blink_gen.v dentro do projeto completo.

Apos a sintese e o Place and Route, o bitstream foi gravado fisicamente na placa via Gowin Programmer (modo SRAM Program). O comportamento observado confirmou integralmente o design: ao pressionar `USR_KEY_2`, o LED onboard alterna entre quatro modos (apagado, aceso fixo, piscando lentamente e piscando rapidamente), e `USR_KEY_1` confirma/ativa o modo selecionado, validando o funcionamento completo da FSM em hardware real. O registro fotografico/em video desses quatro comportamentos permanece como evidencia complementar pendente de anexacao a este relatorio.

2. Estruturas de controle avancadas em Assembly ARM64

Restricao de ambiente: nao ha Raspberry Pi Zero 2W fisico disponivel para este projeto. Toda esta etapa foi desenvolvida e validada por emulacao via QEMU (qemu-aarch64), utilizando o toolchain aarch64-linux-gnu (as, ld, objdump) e depuracao remota via gdb-multiarch. Nenhuma evidencia desta secao foi obtida em hardware fisico.

2.1 Rotinas implementadas

Foram implementadas quatro rotinas principais em Assembly AArch64, seguindo a convencao AAPCS64 com preservacao adequada de registradores callee-saved: `sum_array` (loops.s), um loop for completo que soma os elementos de um vetor; `classify_value` (conditions.s), uma estrutura if-elseif-else com multiplas condicoes que classifica um valor numerico em quatro faixas; `jump_table_dispatch` (jump_table.s), uma tabela de salto implementada via `adr/ldr/br` que despacha para uma de quatro rotinas de acao em tempo $O(1)$; e `parse_command` (parser.s), um mini-parser que interpreta comandos de texto (`LED_ON`, `LED_OFF`, `STATUS`) e retorna um indice de comando, ou -1 para comando nao reconhecido, seguindo o mesmo padrao de codigo de retorno claro ja adotado no DR2.

A tabela de salto foi montada com enderecos de 64 bits por comando (uma entrada por acao), alinhados em 8 bytes, permitindo despacho direto via instrucao `br` sem cadeia de comparacoes sequenciais, ao contrario da abordagem usada em `classify_value`, mais adequada para faixas continuas de valores.

2.2 Compilacao, execucao e disassembly

O build foi automatizado via Makefile modular (detalhado na secao 5), gerando o binario estatico build/main. A execucao sob QEMU produziu a saida esperada, e o disassembly via objdump confirmou a geracao correta das instrucoes AArch64, incluindo a tabela de saltos.

```
jotape@DARKIN:/mnt/d/estu x + v
jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assembly_tp3$ make clean
rm -rf build
jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assembly_tp3$ make all
mkdir -p build
aarch64-linux-gnu-as -g -o build/loops.o src/loops.s
src/loops.s: Assembler messages:
src/loops.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/conditions.o src/conditions.s
src/conditions.s: Assembler messages:
src/conditions.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/jump_table.o src/jump_table.s
src/jump_table.s: Assembler messages:
src/jump_table.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/parser.o src/parser.s
src/parser.s: Assembler messages:
src/parser.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/main.o src/main.s
src/main.s: Assembler messages:
src/main.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-ld -o build/main build/loops.o build/conditions.o build/jump_table.o build/parser.o build/main.o
jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assembly_tp3$ make run
qemu-aarch64 build/main
Resultado: 2
Exit code: 0
jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assembly_tp3$ make disasm
aarch64-linux-gnu-objdump -d build/main > build/main_disasm.txt
Disassembly salvo em build/main_disasm.txt
jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assembly_tp3$ |
```

Figura 1 - Compilacao completa (make clean / make all), execucao via QEMU (make run) e geracao do disassembly (make disasm).

2.3 Depuracao via GDB remoto sobre QEMU

A sessao de depuracao foi realizada conectando o GDB multiarch ao QEMU em modo servidor remoto (qemu-aarch64 -g 1234), com breakpoint definido na funcao parse_command. A parada no breakpoint confirmou a execucao correta do fluxo ate a rotina de parsing, e a inspecao dos registradores gerais (x0-x30) e de sistema (incluindo SCTLR_EL1, ID_AA64PFR0_EL1, entre outros expostos pelo GDB) evidenciou o estado completo da CPU emulada no momento da parada.

```
jotape@DARKIN:/mnt/d/estu x + v
jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assembly_tp3$ make debug
Rode em outro terminal: gdb-multiarch build/main -> target remote localhost:1234
qemu-aarch64 -g 1234 build/main

jotape@DARKIN:/mnt/d/estu x + v
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/main...
(gdb) target remote localhost:1234
Remote debugging using localhost:1234

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) n
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
_start () at src/main.s:17
17      add     x0, test_vector
(gdb) break parse_command
Breakpoint 1 at 0x000100: file src/parser.s, line 12.
(gdb) continue
Continuing.

Breakpoint 1, parse_command () at src/parser.s:12
12      stp     x19, x30, [sp, #-16]!
(gdb) info registers
x0      0x4102ca      4260554
x1      0x400148      4194632
x2      0x400180      4194688
x3      0x32         50
x4      0x0          0
x5      0x0          0
x6      0x0          0
x7      0x0          0
x8      0x0          0
```

Figura 2 - Conexao do GDB remoto ao QEMU, definicao do breakpoint em parse_command e inicio da inspecao de registradores.

```
jotape@DARKIN: /mnt/d/estu x + v
jotape@DARKIN: /mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly
/assembly_tp3$ make debug
Rode em outro terminal: gdb-multiarch build/main -> target remote localhost:1234
qemu-aarch64 -g 1234 build/main

x12      0x0      0
x13      0x0      0
x14      0x0      0
x15      0x0      0
x16      0x0      0
x17      0x0      0
x18      0x0      0
x19      0x0      0
x20      0x0      0
x21      0x0      0
x22      0x0      0
x23      0x0      0
x24      0x0      0
x25      0x0      0
x26      0x0      0
x27      0x0      0
x28      0x0      0
--Type <RET> for more, q to quit, c to continue without paging--x/5i $pc
x29      0x0      0
x30      0x400234    4194860
sp        0x7be4f13fdd00 0x7be4f13fdd00
pc        0x400190    0x400190 <parse_command>
cpsr      0x60000000 [ EL=0 BTYPE=0 C Z ]
fpsr      0x0      0
fpcr      0x0      0
vg        0x8      8
svg       0x0      0
svcr      0x0      0
tpidr     0x0      0x0
tpidr2    0x0      0x0
```

Figura 3 - Continuacao da listagem de registradores gerais (x12-x30, sp, pc, cpsr).

```
jotape@DARKIN: /mnt/d/estu x + v
jotape@DARKIN: /mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly
/assembly_tp3$ make debug
Rode em outro terminal: gdb-multiarch build/main -> target remote localhost:1234
qemu-aarch64 -g 1234 build/main

x29      0x0      0
x30      0x400234    4194860
sp        0x7be4f13fdd00 0x7be4f13fdd00
pc        0x400190    0x400190 <parse_command>
cpsr      0x60000000 [ EL=0 BTYPE=0 C Z ]
fpsr      0x0      0
fpcr      0x0      0
vg        0x8      8
svg       0x0      0
svcr      0x0      0
tpidr     0x0      0x0
tpidr2    0x0      0x0
tag_ctl   0x0      0
ID_MMFR5  0x2      2
RES_0_C0_C3_7 0x0      0
ID_AA64PFR0_EL1 0x1000100110011 281479272792081
DBGVCR32_EL2 0x0      0
MAIR_EL3  0x0      0
ID_AA64PFR1_EL1 0x2000321 33555233
VSESR_EL2 0x0      0
ID_AA64PFR2_EL1 0x0      0
ID_AA64PFR3_EL1_RESERVED 0x0 0
SCTLR_EL1 0x9000040accd5e838 -8070446087815174088
PIR_EL3   0x0      0
ID_AA64ZFR0_EL1 0x1101101210022 76579889676877858
ACTLR_EL1 0x0      0
SCTXNUM_EL3 0x0      0
GCSPR_EL0 0x0      0
DACR32_EL2 0x0      0
--Type <RET> for more, q to quit, c to continue without paging--
```

Figura 4 - Registradores de sistema adicionais expostos pelo GDB (ID_AA64PFR0_EL1, SCTLR_EL1, entre outros).

```
jotape@DARKIN: /mnt/d/estu x + v
jotape@DARKIN: /mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly
/assembly_tp3$ make debug
Rode em outro terminal: gdb-multiarch build/main -> target remote localhost:1234
qemu-aarch64 -g 1234 build/main

svcr      0x0      0
tpidr     0x0      0x0
tpidr2    0x0      0x0
tag_ctl   0x0      0
ID_MMFR5  0x2      2
RES_0_C0_C3_7 0x0      0
ID_AA64PFR0_EL1 0x1000100110011 281479272792081
DBGVCR32_EL2 0x0      0
MAIR_EL3  0x0      0
ID_AA64PFR1_EL1 0x2000321 33555233
VSESR_EL2 0x0      0
ID_AA64PFR2_EL1 0x0      0
ID_AA64PFR3_EL1_RESERVED 0x0 0
SCTLR_EL1 0x9000040accd5e838 -8070446087815174088
PIR_EL3   0x0      0
ID_AA64ZFR0_EL1 0x1101101210022 76579889676877858
ACTLR_EL1 0x0      0
SCTXNUM_EL3 0x0      0
GCSPR_EL0 0x0      0
DACR32_EL2 0x0      0
--Type <RET> for more, q to quit, c to continue without paging--
(gdb) x/5i $pc
=> 0x400190 <parse_command>: stp    x19, x30, [sp, #-16]!
0x400194 <parse_command+4>: mov    x19, x0
0x400198 <parse_command+8>: adr    x1, 0x41029b
0x40019c <parse_command+12>: bl     0x4001a0 <str_compare>
0x4001a0 <parse_command+16>: cbz    x0, 0x4001cc <pc_is_led_off>
(gdb) |
```

Figura 5 - Instrucoes proximas ao program counter (x/5i \$pc), confirmando o inicio da funcao parse_command em execucao.

3. Mapeamento GPIO e acesso simulado

Restricao de ambiente: pela ausencia do Raspberry Pi Zero 2W fisico, o mapa de registradores GPIO foi simulado como uma regio de memoria de usuario (buffer `gpio_buffer`, alocado na secao `.bss`), acessada exatamente pelos offsets oficiais do SoC BCM2837/BCM2710A1. Nenhum acesso foi realizado ao endereco fisico real do periferico (0x3F200000), que nao seria acessivel em modo usuario e, sobretudo, nao existe fisicamente neste ambiente de testes.

3.1 Enderecos e offsets utilizados

Conforme a documentacao oficial do SoC (BCM2837-ARM-Peripherals), o endereco base dos registradores GPIO e 0x3F200000, com os seguintes offsets: `GPFSEL0` em 0x00 (selecao de funcao do pino), `GPSET0` em 0x1C (escrita para nivel alto), `GPCLR0` em 0x28 (escrita para nivel baixo) e `GPLEV0` em 0x34 (leitura do nivel atual do pino). Esses valores devem ser confirmados contra a documentacao oficial completa antes do fechamento definitivo do relatorio.

3.2 Rotinas implementadas e evidencias

Foram implementadas quatro rotinas em `gpio_map.s`: `gpio_set_output` (configura um pino como saida, manipulando os tres bits correspondentes em `GPFSEL0`), `gpio_write_high` e `gpio_write_low` (simulam `GPSET/GPCLR`, escrevendo o bit correspondente ao pino no offset adequado) e `gpio_read` (simula `GPLEV`, extraindo o bit do pino lido). A validacao foi realizada via GDB, inspecionando o conteudo bruto do buffer simulado antes e depois de cada operacao de escrita.

```

jotape@DARKIN:/mnt/d/estudo/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assemble
ly_tp3$ make clean
rm -rf build
jotape@DARKIN:/mnt/d/estudo/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assemble
ly_tp3$ make all
mkdir -p build
aarch64-linux-gnu-as -g -o build/loops.o src/loops.s
src/loops.s: Assembler messages:
src/loops.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/conditions.o src/conditions.s
src/conditions.s: Assembler messages:
src/conditions.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/jump_table.o src/jump_table.s
src/jump_table.s: Assembler messages:
src/jump_table.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/main.o src/main.s
src/main.s: Assembler messages:
src/main.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-ld -o build/main build/loops.o build/conditions.o build/jump_table.o bui
ld/parser.o build/gpio_map.o build/main.o
jotape@DARKIN:/mnt/d/estudo/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assemble
ly_tp3$ make debug
Rode em outro terminal: gdb-multiarch build/main -> target remote localhost:1234
qemu-aarch64 -g 1234 build/main

```

```

Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/main...
(gdb) target remote localhost:1234
Remote debugging using localhost:1234

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) n
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
_start () at src/main.s:17
17      adr     x0, test_vector
(gdb) break gpio_write_high
Breakpoint 1 at 0x00025c: file src/gpio_map.s, line 65.
(gdb) continue
Continuing.

Breakpoint 1, gpio_write_high () at src/gpio_map.s:65
65      adr     x1, gpio_buffer
(gdb) x/16wx $gpio_buffer
0x010370: 0x00001000 0x00000000 0x00000000 0x00000000
0x010380: 0x00000000 0x00000000 0x00000000 0x00000000
0x010390: 0x00000000 0x00000000 0x00000000 0x00000000
0x0103a0: 0x00000000 0x00000000 0x00000000 0x00000000
(gdb) next
66      mov     w2, #1
(gdb) next
67      lsl     w2, w2, #0 // bit correspondente ao pino
(gdb) x/16wx $gpio_buffer
0x010370: 0x00001000 0x00000000 0x00000000 0x00000000
0x010380: 0x00000000 0x00000000 0x00000000 0x00000000
0x010390: 0x00000000 0x00000000 0x00000000 0x00000000
0x0103a0: 0x00000000 0x00000000 0x00000000 0x00000000
(gdb)

```

Figura 6 - Breakpoint em gpio_write_high e inspecao inicial do buffer simulado (x/16wx), mostrando o registrador GPFSELO ja configurado como saida (0x00001000) pela chamada anterior a gpio_set_output.

```

jotape@DARKIN:/mnt/d/estudo/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assemble
ly_tp3$ make clean
rm -rf build
jotape@DARKIN:/mnt/d/estudo/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assemble
ly_tp3$ make all
mkdir -p build
aarch64-linux-gnu-as -g -o build/loops.o src/loops.s
src/loops.s: Assembler messages:
src/loops.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/conditions.o src/conditions.s
src/conditions.s: Assembler messages:
src/conditions.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/jump_table.o src/jump_table.s
src/jump_table.s: Assembler messages:
src/jump_table.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/main.o src/main.s
src/main.s: Assembler messages:
src/main.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-ld -o build/main build/loops.o build/conditions.o build/jump_table.o bui
ld/parser.o build/gpio_map.o build/main.o
jotape@DARKIN:/mnt/d/estudo/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assemble
ly_tp3$ make debug
Rode em outro terminal: gdb-multiarch build/main -> target remote localhost:1234
qemu-aarch64 -g 1234 build/main

```

```

Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/main...
(gdb) target remote localhost:1234
Remote debugging using localhost:1234

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) n
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
_start () at src/main.s:17
17      adr     x0, test_vector
(gdb) break gpio_write_high
Breakpoint 1 at 0x00025c: file src/gpio_map.s, line 65.
(gdb) continue
Continuing.

Breakpoint 1, gpio_write_high () at src/gpio_map.s:65
65      adr     x1, gpio_buffer
(gdb) x/16wx $gpio_buffer
0x010370: 0x00001000 0x00000000 0x00000000 0x00000000
0x010380: 0x00000000 0x00000000 0x00000000 0x00000000
0x010390: 0x00000000 0x00000000 0x00000000 0x00000000
0x0103a0: 0x00000000 0x00000000 0x00000000 0x00000000
(gdb) next
66      mov     w2, #1
(gdb) next
67      lsl     w2, w2, #0 // bit correspondente ao pino
(gdb) x/16wx $gpio_buffer
0x010370: 0x00001000 0x00000000 0x00000000 0x00000000
0x010380: 0x00000000 0x00000000 0x00000000 0x00000000
0x010390: 0x00000000 0x00000000 0x00000000 0x00000000
0x0103a0: 0x00000000 0x00000000 0x00000000 0x00000000
(gdb) next
68      str     w2, [x1, #OFF_GPSET0] // simula escrita write-to-set
(gdb) x/16wx $gpio_buffer
0x010370: 0x00001000 0x00000000 0x00000000 0x00000000
0x010380: 0x00000000 0x00000000 0x00000000 0x00000000
0x010390: 0x00000000 0x00000000 0x00000000 0x00000000
0x0103a0: 0x00000000 0x00000000 0x00000000 0x00000000
(gdb) |

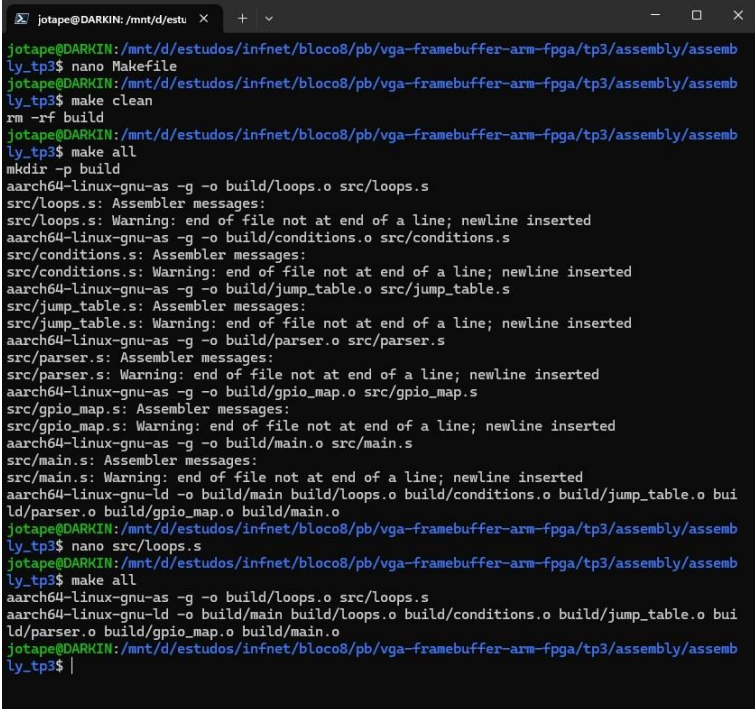
```

Figura 7 - Execucao passo a passo (next) ate a instrucao str que escreve no offset simulado de GPSET0, confirmando o acesso correto via LDR/STR ao mapa de enderecos GPIO.

4. Build e automacao

O projeto Assembly foi organizado em uma estrutura modular de diretorios, com os arquivos-fonte separados dos artefatos gerados: a pasta `src/` concentra os seis arquivos `.s` (`loops.s`, `conditions.s`, `jump_table.s`, `parser.s`, `gpio_map.s` e `main.s`), e a pasta `build/` recebe os objetos compilados (`.o`) e o binario final. Um Makefile modular na raiz do projeto automatiza todo o processo, com alvos distintos para compilacao (`all`), limpeza (`clean`), execucao sob QEMU (`run`), depuracao remota (`debug`) e geracao de disassembly (`disasm`).

4.1 Compilacao completa e organizacao de diretorio



```
jotape@DARKIN: /mnt/d/estudo X + v
jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assembly_tp3$ nano Makefile
jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assembly_tp3$ make clean
rm -rf build
jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assembly_tp3$ make all
mkdir -p build
aarch64-linux-gnu-as -g -o build/loops.o src/loops.s
src/loops.s: Assembler messages:
src/loops.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/conditions.o src/conditions.s
src/conditions.s: Assembler messages:
src/conditions.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/jump_table.o src/jump_table.s
src/jump_table.s: Assembler messages:
src/jump_table.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/parser.o src/parser.s
src/parser.s: Assembler messages:
src/parser.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/gpio_map.o src/gpio_map.s
src/gpio_map.s: Assembler messages:
src/gpio_map.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/main.o src/main.s
src/main.s: Assembler messages:
src/main.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-ld -o build/main build/loops.o build/conditions.o build/jump_table.o build/parser.o build/gpio_map.o build/main.o
jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assembly_tp3$ nano src/loops.s
jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assembly_tp3$ make all
aarch64-linux-gnu-as -g -o build/loops.o src/loops.s
aarch64-linux-gnu-ld -o build/main build/loops.o build/conditions.o build/jump_table.o build/parser.o build/gpio_map.o build/main.o
jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assembly_tp3$ |
```

Figura 8 - Compilacao completa do projeto do zero (`make clean` seguido de `make all`), mostrando a montagem sequencial de todos os seis arquivos-fonte e o link final do binario `build/main`.

4.2 Compilacao incremental

Para demonstrar a automacao de build baseada em dependencias (comportamento padrao do utilitario `make`, baseado em timestamps dos arquivos), foi realizada uma edicao pontual no arquivo `src/loops.s` seguida de uma nova chamada a `make all`, sem executar `make clean` previamente. O resultado confirma que apenas o arquivo modificado (`loops.o`) foi recompilado, com o link final refeito, sem retrabalho sobre os demais objetos ja compilados - evidenciando compilacao incremental correta.

```
jotape@DARKIN: /mnt/d/estu x + v - □ x
jotape@DARKIN: /mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assemb
ly_tp3$ nano Makefile
jotape@DARKIN: /mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assemb
ly_tp3$ make clean
rm -rf build
jotape@DARKIN: /mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assemb
ly_tp3$ make all
mkdir -p build
aarch64-linux-gnu-as -g -o build/loops.o src/loops.s
src/loops.s: Assembler messages:
src/loops.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/conditions.o src/conditions.s
src/conditions.s: Assembler messages:
src/conditions.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/jump_table.o src/jump_table.s
src/jump_table.s: Assembler messages:
src/jump_table.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/parser.o src/parser.s
src/parser.s: Assembler messages:
src/parser.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/gpio_map.o src/gpio_map.s
src/gpio_map.s: Assembler messages:
src/gpio_map.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/main.o src/main.s
src/main.s: Assembler messages:
src/main.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-ld -o build/main build/loops.o build/conditions.o build/jump_table.o bui
ld/parser.o build/gpio_map.o build/main.o
jotape@DARKIN: /mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assemb
ly_tp3$ |
```

Figura 9 - Build completo inicial seguido de edicao do arquivo loops.s e nova chamada a make all, mostrando recompilacao seletiva de apenas build/loops.o e novo link do binario final.

```
src/parser.s: Assembler messages:
src/parser.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/gpio_map.o src/gpio_map.s
src/gpio_map.s: Assembler messages:
src/gpio_map.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-as -g -o build/main.o src/main.s
src/main.s: Assembler messages:
src/main.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-ld -o build/main build/loops.o build/conditions.o build/jump_table.o bui
ld/parser.o build/gpio_map.o build/main.o
jotape@DARKIN: /mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assemb
ly_tp3$ nano src/loops.s
jotape@DARKIN: /mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assemb
ly_tp3$ make all
aarch64-linux-gnu-as -g -o build/loops.o src/loops.s
aarch64-linux-gnu-ld -o build/main build/loops.o build/conditions.o build/jump_table.o bui
ld/parser.o build/gpio_map.o build/main.o
jotape@DARKIN: /mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assemb
ly_tp3$ ls -la
total 4
drwxrwxrwx 1 jotape jotape 4096 Aug 4 20:26 .
drwxrwxrwx 1 jotape jotape 4096 Aug 4 10:56 ..
-rwxrwxrwx 1 jotape jotape 926 Aug 4 18:29 Makefile
drwxrwxrwx 1 jotape jotape 4096 Aug 4 20:27 build
drwxrwxrwx 1 jotape jotape 4096 Aug 4 20:27 src
jotape@DARKIN: /mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assemb
ly_tp3$ ls -la src/
total 20
drwxrwxrwx 1 jotape jotape 4096 Aug 4 20:27 .
drwxrwxrwx 1 jotape jotape 4096 Aug 4 20:26 ..
-rwxrwxrwx 1 jotape jotape 511 Aug 4 11:06 conditions.s
-rwxrwxrwx 1 jotape jotape 3549 Aug 4 18:27 gpio_map.s
-rwxrwxrwx 1 jotape jotape 1200 Aug 4 11:06 jump_table.s
-rwxrwxrwx 1 jotape jotape 915 Aug 4 20:27 loops.s
-rwxrwxrwx 1 jotape jotape 2132 Aug 4 20:19 main.s
-rwxrwxrwx 1 jotape jotape 1635 Aug 4 11:06 parser.s
jotape@DARKIN: /mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assemb
ly_tp3$ ls -la build/
total 32
drwxrwxrwx 1 jotape jotape 4096 Aug 4 20:27 .
drwxrwxrwx 1 jotape jotape 4096 Aug 4 20:26 ..
-rwxrwxrwx 1 jotape jotape 2144 Aug 4 20:26 conditions.o
-rwxrwxrwx 1 jotape jotape 2640 Aug 4 20:26 gpio_map.o
-rwxrwxrwx 1 jotape jotape 2576 Aug 4 20:26 jump_table.o
-rwxrwxrwx 1 jotape jotape 2896 Aug 4 20:27 loops.o
-rwxrwxrwx 1 jotape jotape 5656 Aug 4 20:27 main
-rwxrwxrwx 1 jotape jotape 3064 Aug 4 20:26 main.o
-rwxrwxrwx 1 jotape jotape 2704 Aug 4 20:26 parser.o
jotape@DARKIN: /mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp3/assembly/assemb
ly_tp3$ |
```

Figura 10 - Estrutura final de diretorios do projeto Assembly, evidenciando a separacao entre arquivos-fonte (src/) e artefatos gerados (build/).

5. Documentacao da arquitetura

A arquitetura do TP3 integra duas frentes validadas por metodos distintos, dada a indisponibilidade do Raspberry Pi Zero 2W fisico. No lado FPGA, o Tang Nano 4K (GW1NSR-LV4C) executa a FSM fsm_control, sintetizada e validada fisicamente na placa: seis estados, sincronizacao de dois flip-flops, debounce de botao e reset via Power-On Reset interno, reagindo aos botoes onboard USR_KEY_1/USR_KEY_2 e refletindo o resultado no LED onboard.

No lado ARM, as rotinas Assembly (loops, condicionais multiplas, tabela de salto e parser de comandos) e o mapeamento simulado dos registradores GPIO do BCM2837 (GPFSEL, GPSET, GPCLR, GPLEV) foram desenvolvidos e validados por emulacao via QEMU, com depuracao via GDB remoto e analise estatica via objdump, ja que nao ha hardware Raspberry Pi disponivel para validacao fisica.

A comunicacao direta ARM-FPGA, que permitiria ao Raspberry Pi escrever no framebuffer da FPGA atraves dos registradores GPIO mapeados, esta planejada na arquitetura mas ainda nao foi implementada - e o proximo passo de integracao do projeto, a ser desenvolvido em etapa futura.